

3D Printer Factory Simulator

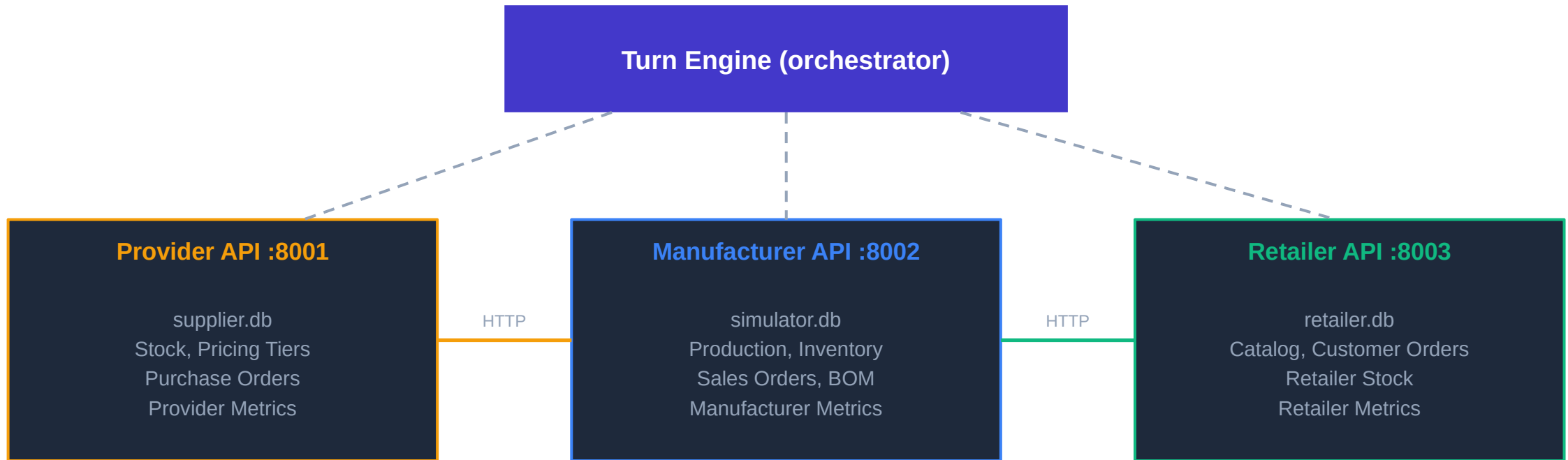
Week 8 — Autonomy & Analysis

A discrete-event supply-chain simulation with three autonomous AI agents

Multi-Agent Coordination | Scenario-Driven Stress Testing | Emergent Behaviour

System Architecture

Three independent FastAPI apps orchestrated by a Turn Engine



No app touches another's database. All coordination happens through HTTP + shared world state.

3 SQLite DBs | 3 FastAPI apps | 1 Turn Engine | Scenario-driven events

Agent: Provider Manager

skills/provider-manager.md — Manages raw material stock & pricing

Decision Framework

- 1. Assess — query stock levels & pending orders
- 2. Restock — replenish products below 50% of starting level
- 3. Adjust Prices — raise if stock < 30%, lower if > 150%
- 4. Summarise — structured log for engine capture

Market Signal Response

- `supply_modifier < 0.7` → raise prices, conserve stock
- `demand_modifier > 1.5` → build stock ahead of surge
- Max 15% price change per day (gradual adjustments)
- Never let stock hit zero if orders are pending

Available CLI Commands

- `./provider-cli stock` | `./provider-cli orders list` | `./provider-cli catalog`
- `./provider-cli restock <product> <qty>` | `./provider-cli price set <product> <tier> <price>`
- Agent NEVER calls day advance — Turn Engine is the sole orchestrator

Agent: Manufacturer Manager

skills/manufacturer-manager.md — Production planning & material procurement

Decision Framework (6 steps)

- 1. Assess — day, wallet, stock, capacity, sales orders
- 2. Identify Bottlenecks — compare BOM vs inventory
- 3. Release Production — prioritize older orders first
- 4. Plan Material Orders — bulk buy for tier discounts
- 5. Adjust Pricing — raise if wallet < 3000
- 6. Report — structured summary for logging

Key Constraints

- Daily production capacity: 10 units (configurable)
- Must check wallet before ordering materials
- Must check supplier stock before placing PO
- Tier pricing: 10+ = -10%, 50+ = -18%, 100+ = -25%
- Materials reserved on release, consumed on completion

Market Signal Response

- $\text{demand_modifier} > 1.0 \rightarrow$ keep stock ready, consider price increases
- $\text{demand_modifier} < 1.0 \rightarrow$ conserve cash, reduce orders, avoid overproduction

Agent: Retail Manager

skills/retail-manager.md — Customer fulfillment & pricing strategy

Decision Framework

- 1. Fulfill — ship from stock or backorder each pending order
- 2. Reorder — purchase from manufacturer if stock < 3 days demand
- 3. Price — raise 5% if stock low, lower 5% if piling up
- 4. Summarise — orders fulfilled, backordered, purchases placed

Key Rules

- NEVER leave customer orders in 'pending' status
- Retail price MUST be > manufacturer wholesale + 20%
- Reorder enough to cover ~5 days of expected demand
- Backordered orders trigger extra reorder quantity

Market Signal Response

- $\text{demand_modifier} > 1.5$ → place larger purchase orders now, hold or raise prices
- $\text{demand_modifier} < 0.8$ → slow reorders, consider cutting prices 5%
- $\text{price_sensitivity} = \text{'high'}$ → be cautious raising prices, customers are shopping around

Scenario Design

Two scenarios: calm baseline vs. volatile holiday rush

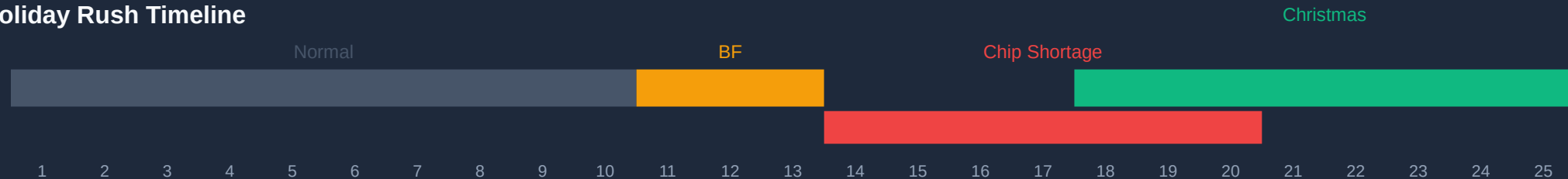
Calm Market (Control)

- 25 days, stable demand (mean=4, var=1)
- demand_modifier = 1.0 throughout
- supply_modifier = 1.0 throughout
- Purpose: establish baseline agent behaviour
- No disruptions — tests steady-state decisions

Holiday Rush (Volatile)

- Days 1-10: Normal (demand=1.0x, supply=1.0x)
- Days 11-13: Black Friday (demand=3.0x, price-sensitive)
- Days 14-20: Chip Shortage (supply=0.4x, lead time=2.0x)
- Days 18-25: Christmas (demand=2.5x, supply=0.6x)
- Days 18-20: COMPOUND (demand=3.75x, supply=0.24x)

Holiday Rush Timeline



Live Demo

Running 2-3 simulated days with all three agents

```
python turn_engine.py config/sim.json scenarios/holiday-rush.json 25
```

What to Watch

- Agent decisions printed per turn
- Market signals passed to each agent
- Day summary: orders / fulfilled / backordered
- Wallet changes and inventory shifts

Emergent Behaviour

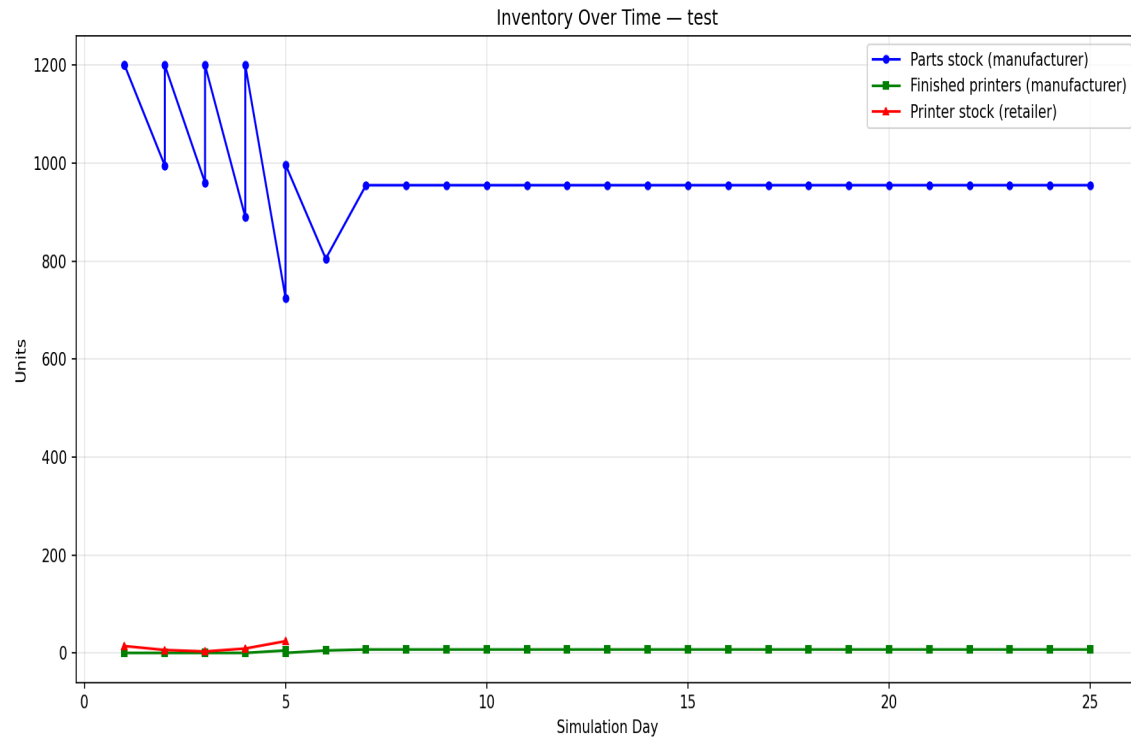
- Bullwhip effect: demand amplified upstream
- Price wars during low demand
- Stockout cascades across the chain
- Coordination WITHOUT direct communication

Dashboard

- localhost:8002 — full React SPA
- Real-time SVG charts
- Scenario timeline overlay
- Event logs from all 3 apps

The demo is the most important part. Have a plan for what to do if agents stall.

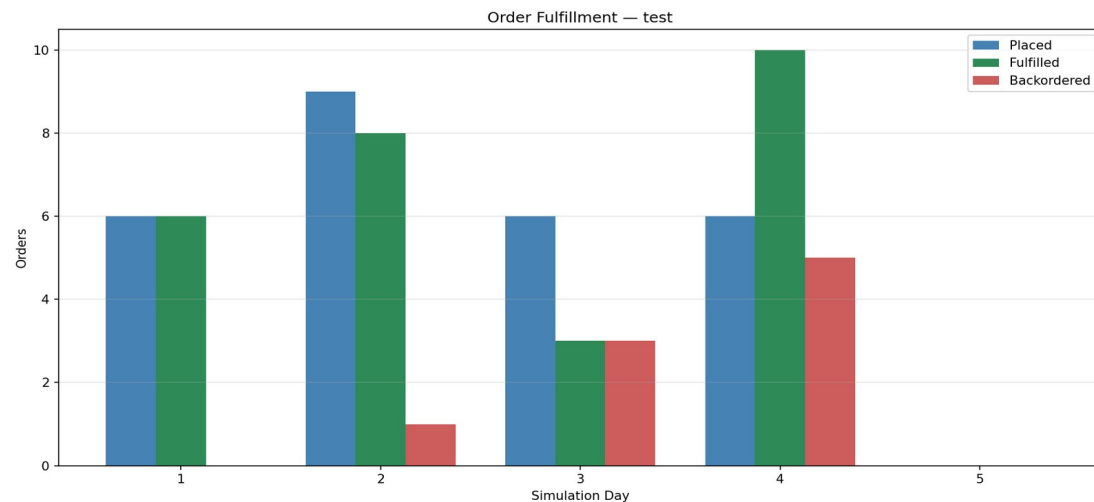
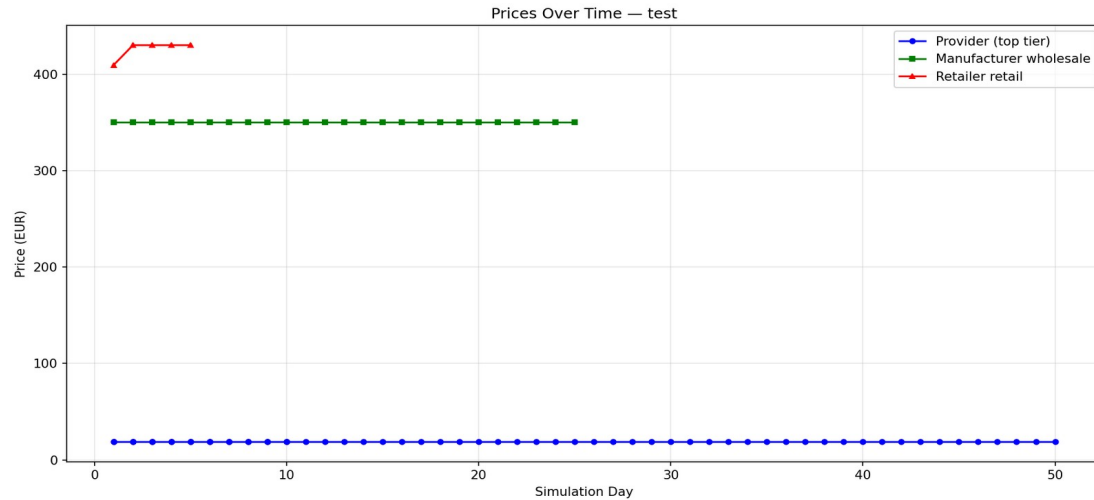
Results: Inventory Over Time



Interpretation

- Three lines: parts stock (manufacturer), finished-printer stock (manufacturer), printer stock (retailer)
- Look for: stock drawdowns during Black Friday (days 11-13), recovery patterns after the spike
- Chip shortage (days 14-20) should cause parts stock to plateau or drop as supply is constrained
- Christmas overlap (days 18-20) is the stress peak — compound demand 3.75x with supply at 0.24x
- Did the manufacturer build stock ahead of Black Friday? If not, stockouts cascade downstream

Results: Prices & Order Fulfillment



Prices Interpretation

- Three lines: provider part price, manufacturer wholesale, retailer retail
- Prices should rise during shortage (days 14-20) as agents react to $\text{supply_modifier} < 0.7$
- Did prices stabilise or oscillate? Oscillation = agents overreacting to signals
- Price sensitivity during Black Friday: did the retailer hold prices despite high demand?

Fulfillment Interpretation

- Daily bars: orders placed vs fulfilled vs backordered
- Backorder spikes indicate stockouts — whose decision was the root cause?
- Compare calm scenario (near-100% fulfillment) vs holiday rush (gaps expected)
- Bullwhip moment: demand variance amplified upstream through the chain

Reflection & Lessons Learned

What Worked

- Three-DB isolation: clean boundaries, no coupling
- Skill-based agent design: version-controlled, testable
- Scenario-driven testing: reproducible stress conditions
- Event-sourced logging: full audit trail for analysis

Challenges

- Agents can make bad calls that cascade into crises
- Skill ambiguities revealed mid-run (tuning needed)
- Compound events (3.75x demand + 0.24x supply) are brutal
- Observability is crucial — without metrics, analysis is impossible

Key Takeaway

- Emergence is the feature: coordination happens through world state, not direct communication
- Deterministic software handles execution; LLMs handle strategy; well-defined interfaces keep everything auditable
- The hardest part was not any single component — it was keeping your head across all of them
- This architecture mirrors real autonomous systems: Uber surge pricing, Amazon warehouse robots, MMO economies

This is not a toy. This is the shape of real autonomous business systems.